

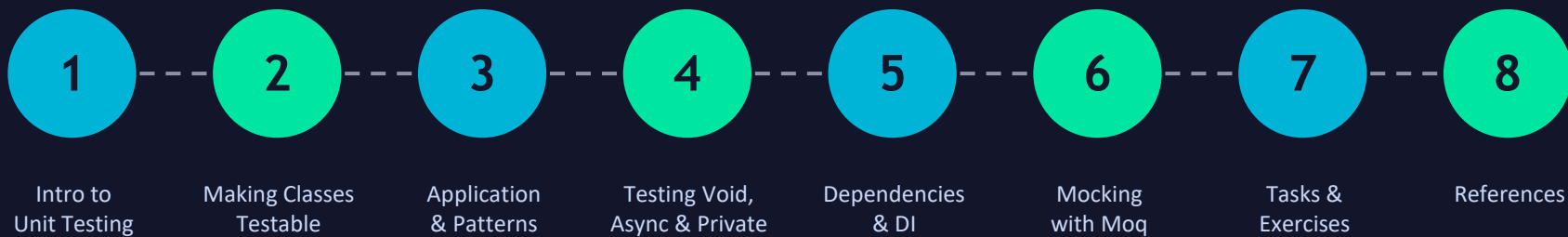


Unit Testing

with xUnit | .NET 10

Waleed Elwakeel

Course Roadmap





1

Introduction to Unit Testing

1.1 What is Unit Testing?

Unit tests are **low-level tests** that focus on verifying a **small, isolated part** of the software system — typically a single method or behavior of a class.



Fast

Run in milliseconds



Isolated

No external dependencies



Repeatable

Same result every time



Self-Validating

Pass or fail automatically

1.2 Types of Testing

Unit Testing

Tests a single function or method in isolation.
Uses test cases, mock dependencies.

Integration Testing

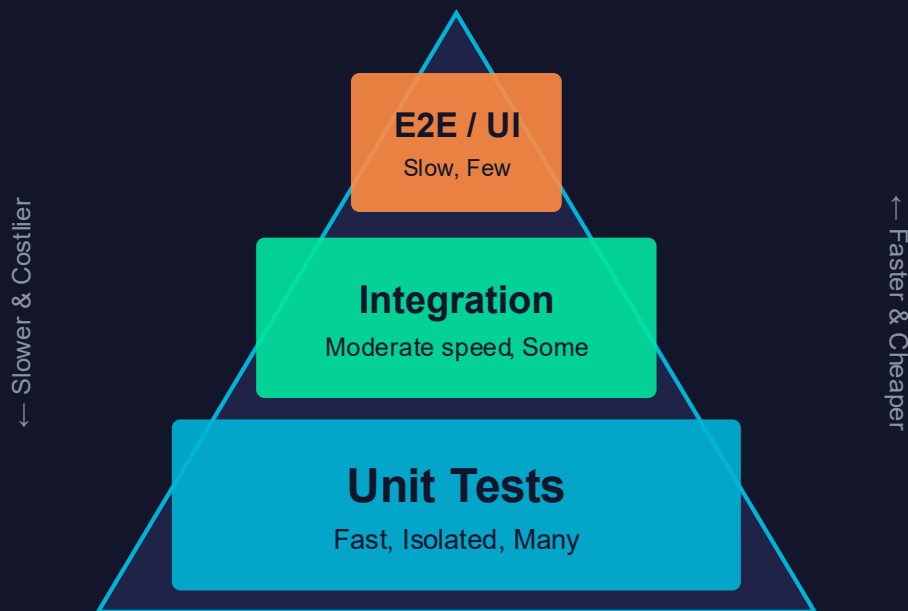
Tests multiple components working together.
Verifies interactions and data flows.

Acceptance Testing

End-to-end validation done by or for the client.
Confirms business requirements are met.

Source: martinfowler.com/bliki/TestPyramid.html

1.3 The Test Pyramid



Key Insight

Write many unit tests

They are fast, cheap, and catch bugs early.

Fewer integration tests

Verify component interactions.

Fewest E2E tests

Expensive, brittle, but validate full workflows.

Source: martinfowler.com/bliki/TestPyramid.html

1.4 Why Use Unit Testing?

Validate Behavior

Confirm each method does what it should — correct output, correct data types, correct side-effects.

Safe Refactoring

Add or modify features without unintentionally breaking existing functionality.

Living Documentation

Tests describe how the code is intended to work — better than comments.

Faster Than Manual

Automated tests run in seconds vs. minutes of manual testing per change.



2

Making Classes Testable

2.1 The Problem: Tight Coupling

Tightly Coupled (Not Testable)

```
public class OrderService
{
    public void PlaceOrder(Order order)
    {
        // Hard dependency!
        var repo = new SqlOrderRepo();
        repo.Save(order);
        var mailer = new SmtplibMailer();
        mailer.Send(order.Email, "...");
    }
}
```

Loosely Coupled (Testable)

```
public class OrderService
{
    private readonly IOrderRepo _repo;
    private readonly IMailer _mailer;

    public OrderService(
        IOrderRepo repo, IMailer mailer)
    {
        _repo = repo;
        _mailer = mailer;
    }
}
```

Source: docs.microsoft.com/aspnet/core/fundamentals/dependency-injection | danclarke.com/writing-testable-code

2.2 How to Make a Class Testable

1

Depend on Abstractions

Use interfaces (IService) not concrete classes (SqlService). Follows the Dependency Inversion Principle.

2

Use Constructor Injection

Pass all dependencies via the constructor. Makes dependencies explicit and visible.

3

Avoid "new" Inside Logic

Don't instantiate dependencies inside methods. Let the DI container handle object creation.

4

Single Responsibility

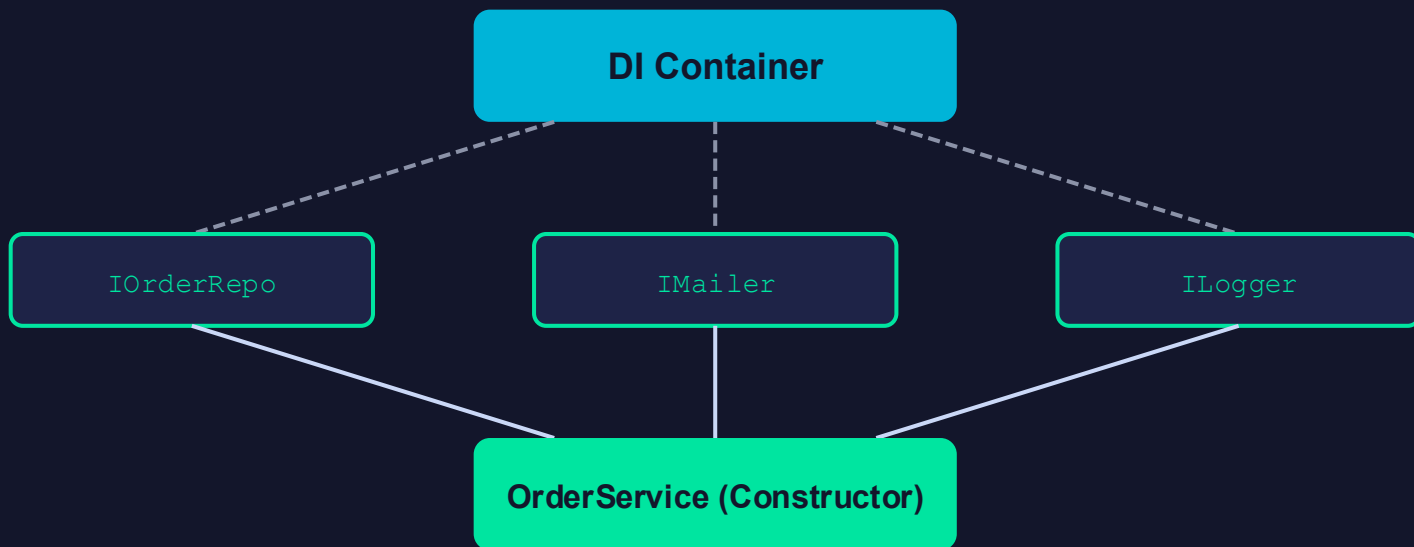
Each class should do one thing. Smaller classes = easier to test with fewer mocks.

5

Register in DI Container

In Program.cs: `builder.Services.AddScoped<IOrderRepo, SqlOrderRepo>();`

2.3 Dependency Injection Flow



In tests, you replace real implementations with mocks (Moq) — the class under test doesn't know the difference.



Application & Patterns

3.1 Naming Convention

Test Class Naming

ClassName + Tests → e.g. StudentService.cs → StudentServiceTests.cs

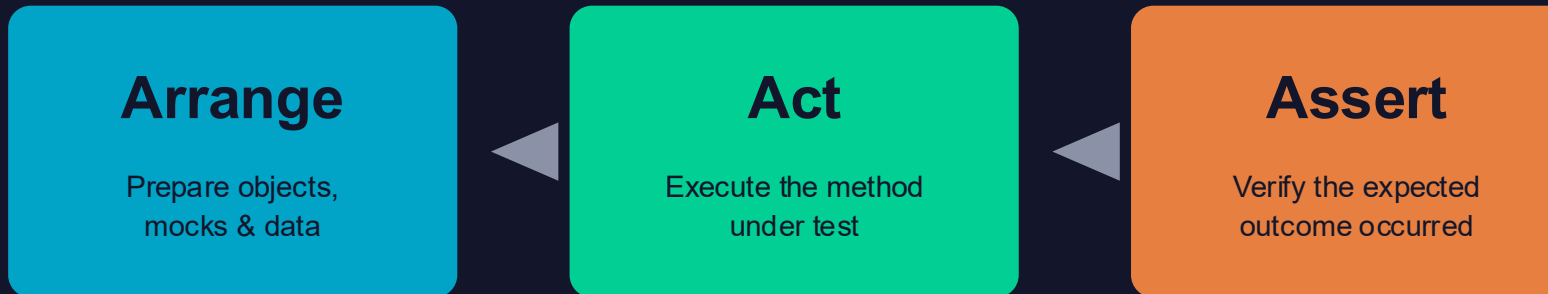
Pattern 1: MethodName_State_Expected

AdmitStudent_IfEmailMissing_FailToAdmit
GetOrder_WithValidId_ReturnsOrder
Calculate_NegativeInput_ThrowsException

Pattern 2: Given_When_Then

GivenValidEmail_WhenAdmit_ThenSuccess
GivenNoStock_WhenOrder_ThenThrows
GivenAdmin_WhenDelete_ThenRemoved

3.2 Testing Pattern – AAA



```
[Fact]
public void GetStudent_WithValidId_ReturnsStudent()
{
    // Arrange
    var repo = new Mock<IStudentRepo>();
    repo.Setup(r => r.GetById(1)).Returns(new Student { Id = 1, Name = "Ali" });
    var svc = new StudentService(repo.Object);

    // Act
    var result = svc.GetStudent(1);

    // Assert
    Assert.NotNull(result);
    Assert.Equal("Ali", result.Name);
}
```

3.3 Rules of Unit Testing

- Test each function independently — one unit of work per test.
- Each test should have one logical path (no if/else branching inside tests).
- Tests must not depend on other tests or external state.
- Avoid logic in tests — no loops, conditionals, or try/catch in test code.
- Use clear naming conventions (MethodName_State_Expected).
- One logical assertion per test — multiple Assert calls are OK if they verify the same behavior.

3.4 xUnit Assert – Object Assertions

xUnit Method	Usage
Assert.Equal(exp, act)	Checks value equality
Assert.NotEqual(exp, act)	Checks values are not equal
Assert.Same(exp, act)	Same object reference
Assert.NotSame(exp, act)	Different object references
Assert.IsType<T>(obj)	Exact type match
Assert.IsNotType<T>(obj)	Not that exact type
Assert.IsAssignableFrom<T>(obj)	Checks assignable (incl. inheritance)
Assert.Null(obj)	Object is null
Assert.NotNull(obj)	Object is not null
Assert.True(cond)	Condition is true
Assert.False(cond)	Condition is false
Assert.InRange(val, lo, hi)	Value within range (xUnit only)

3.5 xUnit Assert – String Assertions

xUnit Method	Usage
<code>Assert.Contains(sub, str)</code>	String contains substring
<code>Assert.DoesNotContain(sub, str)</code>	String does not contain substring
<code>Assert.StartsWith(exp, str)</code>	String starts with prefix
<code>Assert.EndsWith(exp, str)</code>	String ends with suffix
<code>Assert.Matches(regex, str)</code>	String matches regex pattern
<code>Assert.DoesNotMatch(regex, str)</code>	String does not match regex
<code>Assert.Equal(exp, act, ignoreCase: true)</code>	Case-insensitive comparison
<code>Assert.Empty(str)</code>	String is empty (xUnit only)

3.6 xUnit Assert – Collection Assertions

xUnit Method	Usage
<code>Assert.Contains(item, col)</code>	Collection contains element
<code>Assert.DoesNotContain(item, col)</code>	Collection doesn't contain element
<code>Assert.All(col, predicate)</code>	All items match a condition
<code>Assert.Empty(col)</code>	Collection is empty
<code>Assert.NotEmpty(col)</code>	Collection is not empty
<code>Assert.Single(col)</code>	Collection has exactly one item
<code>Assert.Equal(exp, act)</code>	Equal elements, same order, same count
<code>Assert.Equivalent(exp, act)</code>	Same elements, any order, same count
<code>Assert.Subset(super, sub)</code>	Collection is subset of superset
<code>Assert.Collection(col, ...)</code>	Per-element assertion inspectors
<code>Assert.Distinct(col)</code>	All items are unique

3.7 xUnit Attributes

xUnit	Usage
[Fact]	Marks a test method (no parameters)
[Theory]	Data-driven test (parameterized)
[InlineData(...)]	Inline test data for [Theory]
[MemberData(...)]	Data from a property/method
[ClassData(...)]	Data from a class implementing IEnumerable
[Trait("Category","X")]	Categorize / group tests
[Trait("Priority","1")]	Set test priority
Assert.Throws<T>()	Test exceptions (inline, not attribute!)
Constructor	Per-test setup
IDisposable.Dispose()	Per-test cleanup
IClassFixture<T>	One-time setup per test class
ICollectionFixture<T>	Shared fixture across test classes
Skip = "reason"	Skip a test with reason

3.8 Data-Driven Tests with [Theory]

```
[Theory]
[InlineData(2, 3, 5)]
[InlineData(-1, 4, 3)]
[InlineData(0, 0, 0)]
public void Add_WithVariousInputs_ReturnsExpectedSum(int a, int b, int expected)
{
    // Arrange
    var calc = new Calculator();

    // Act
    var result = calc.Add(a, b);

    // Assert
    Assert.Equal(expected, result);
}
```

Each [InlineData] runs as a separate test case. Use [MemberData] or [ClassData] for complex data sets.



4

Testing Void, Async & Private

4.1 How to Test Void Methods

Void methods have no return value — test their side-effects and behavior instead.

```
[Fact]
public void SendWelcomeEmail_ValidUser_CallsMailerOnce()
{
    // Arrange
    var mockMailer = new Mock<IMailer>();
    var svc = new UserService(mockMailer.Object);

    // Act (void method – no return value)
    svc.SendWelcomeEmail(new User { Email = "test@x.com" });

    // Assert – verify the SIDE-EFFECT
    mockMailer.Verify(m => m.Send(
        It.Is<string>(e => e == "test@x.com"),
        It.IsAny<string>()
    ), Times.Once);
}
```

4.2 How to Test Async Methods

xUnit natively supports async Task test methods — just use async/await as normal.

```
[Fact]
public async Task GetOrderAsync_ValidId_ReturnsOrder()
{
    // Arrange
    var mockRepo = new Mock<IOrderRepo>();
    mockRepo.Setup(r => r.GetByIdAsync(1))
        .ReturnsAsync(new Order { Id = 1, Total = 99.99m });
    var svc = new OrderService(mockRepo.Object);

    // Act
    var result = await svc.GetOrderAsync(1);

    // Assert
    Assert.NotNull(result);
    Assert.Equal(99.99m, result.Total);
}
```

4.3 Testing Async Exceptions

```
[Fact]
public async Task GetOrderAsync_InvalidId_ThrowsNotFoundException()
{
    var mockRepo = new Mock<IOrderRepo>();
    mockRepo.Setup(r => r.GetByIdAsync(-1)).ThrowsAsync(new NotFoundException());
    var svc = new OrderService(mockRepo.Object);

    await Assert.ThrowsAsync<NotFoundException>(() => svc.GetOrderAsync(-1));
}
```

MSTest (Old)

[ExpectedException(typeof(...))] attribute.

Doesn't specify which line throws.

Can hide bugs in Arrange code.

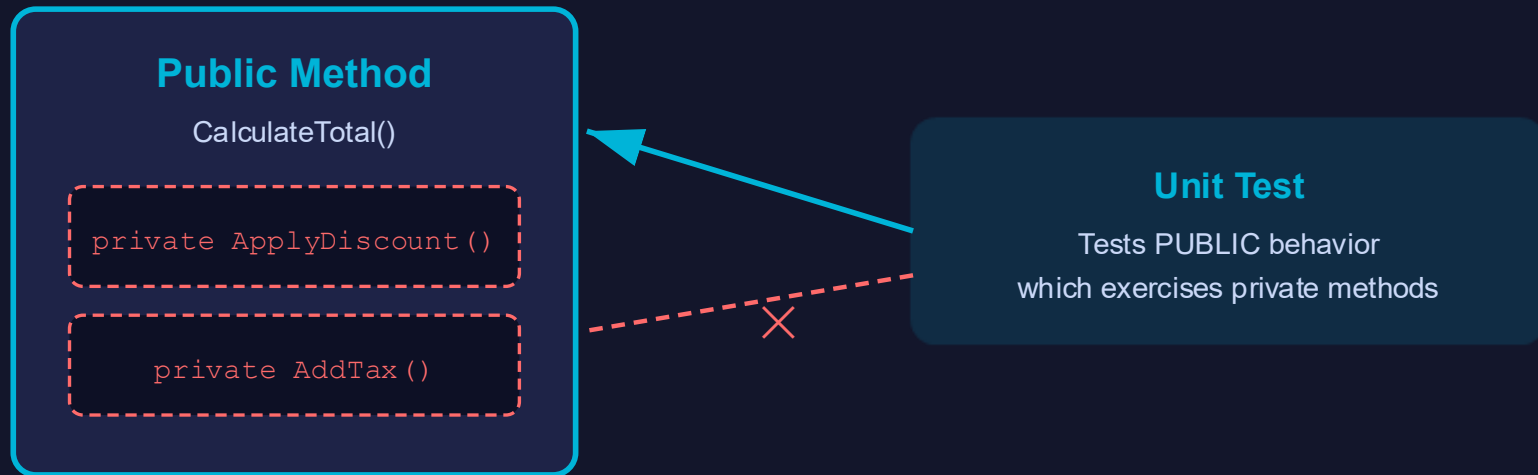
xUnit (Modern)

Assert.Throws<T>() wraps exact code.

Pinpoints the throwing line precisely.

Returns the exception for further asserts.

4.4 Private Methods: Don't Test Directly



Private methods are implementation details. **Test the public method's observable behavior** — if the public method works correctly, its private helpers are exercised implicitly. If a private method is complex enough to need direct tests, it likely belongs in its own class.

5

Dependencies & DI

5.1 What Are Dependencies?

A dependency exists when a class **can't perform its behavior** without another class. Tight coupling between dependencies makes code **hard to test, reuse, and maintain**.

DIP — Dependency Inversion Principle

High-level modules should not depend on low-level modules. Both should depend on abstractions.

DI — Dependency Injection

Creation and binding of dependencies is done outside the dependent class (usually via constructor).

IoC — Inversion of Control

A framework/container controls object creation and lifecycle, not your application code.



6

Mocking with Moq

6.1 Mocking – Isolate Your Tests

Mocking generates fake dependencies so tests remain isolated and not brittle.



```
// Arrange
var mockRepo = new Mock<IStudentRepo>();
mockRepo.Setup(r => r.GetById(1)).Returns(new Student { Id = 1 });

var svc = new StudentService(mockRepo.Object); // inject the fake

// Act & Assert
var result = svc.GetStudent(1);
Assert.Equal(1, result.Id);
```



Tasks & Exercises

7.1 Required Tasks

Object Assertions

4 unit tests using `Assert.Equal`, `Assert.Null`, `Assert.True`, `Assert.IsType`, etc.

String Assertions

2 unit tests using `Assert.Contains`, `Assert.StartsWith`, `Assert.Matches`, etc.

Collection Assertions

2 unit tests using `Assert.Contains(col)`, `Assert.All`, `Assert.Single`, etc.

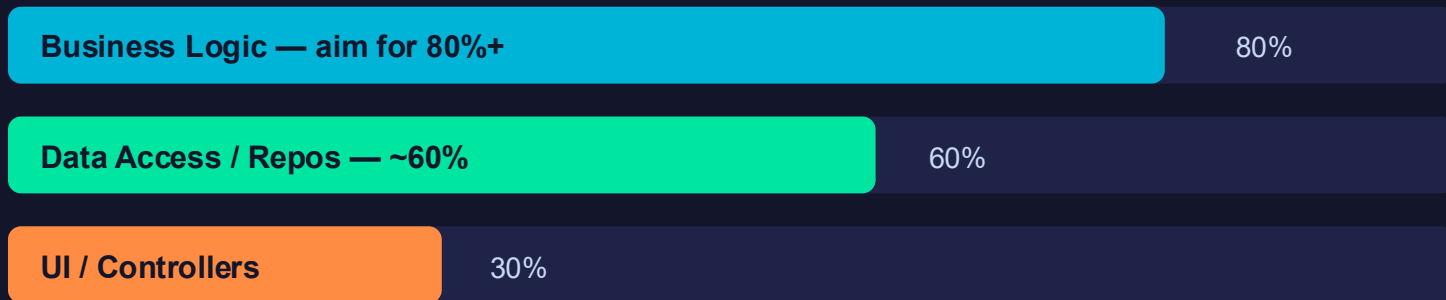
Mocking (Moq)

3 test methods on Repository classes (using Moq).
3 test methods on Service classes (using Moq).

7.2 Code Coverage

Code coverage measures the **percentage of code paths exercised by tests**. It helps find untested code but is not a quality metric — 100% coverage doesn't mean zero bugs.

Coverage Targets



Source: martinfowler.com/bliki/TestCoverage.html

8

References

References

- xunit.net — Official xUnit Documentation
- xunit.net/docs/comparisons — MSTest vs xUnit Comparison
- martinfowler.com/bliki/UnitTest.html — Unit Test Definition
- martinfowler.com/bliki/TestPyramid.html — Test Pyramid
- docs.microsoft.com/dotnet/core/testing/unit-testing-best-practices
- docs.microsoft.com/aspnet/core/fundamentals/dependency-injection
- martinfowler.com/articles/injection.html — Dependency Injection
- danclarke.com/writing-testable-code — Making Code Testable
- github.com/moq/moq — Moq Mocking Framework
- martinfowler.com/bliki/TestCoverage.html — Code Coverage

Thanks!

ANY QUESTIONS?

You can find me at

[linkedin.com/in/waleedelwakeel](https://www.linkedin.com/in/waleedelwakeel)

Waleed_elwakeel13@hotmail.com